

MODULE 11 · DOMAIN 5 - BEYOND SQL

# Something Has To Run It

Orchestration failures are correctness failures wearing a scheduling costume.

DOMAIN 5 of 5    MODULE 11 of 11    PROMPTS 3

A pipeline that fails is fine. A pipeline that succeeds on **stale** input is not.

# Three ways orchestration produces wrong data

- ◆ **Order.** A task runs before its input is ready and reads what was there yesterday.
- ◆ **Retries.** A task runs twice and appends twice. Nothing errors.
- ◆ **Backfill.** A rerun for last month uses today's lookup values.
- ◆ **All three succeed.** Green in the interface, wrong in the warehouse.

## THE POINT

Every one of these looks identical to a healthy run from the scheduler's point of view.

The dependency graph is not the data  
dependency graph.

# The edge nobody drew

Three tasks. Two declared dependencies.

```
01 load_parts >> load_usage >> build_report
02
03 # But inside build_report:
04 spark.table("fs_dw.dim_site_flat")
05
06 # dim_site_flat is loaded by a DAG in another repository,
07 # on its own schedule, roughly 40 minutes later.
08 # There is no dependency between them at all.
09
10 # Most days it wins the race. Some days it does not.
```

## THE RULE

Reading a table is a dependency whether or not anybody declared it.

# What you actually type

Two graphs. Ask for the difference.

Below is a DAG definition and the code for every task in it.

Build two lists and compare them.

1. Dependencies DECLARED in the DAG
2. Every table, path or topic each task READS

Then: any object read by a task where nothing upstream in this DAG writes it. Those are undeclared dependencies.

For each, say what the task would read if it ran early.

Do not assume an external system is safe because it is external.

WHERE YOU HAVE SEEN THIS

This is the reconciliation prompt again. Two descriptions of the same thing, diffed.

Ask what happens when it runs **twice**.

# Why retries are dangerous by default

- ◆ **Retries are on.** Usually two or three, configured once and forgotten.
- ◆ **Failure is rarely clean.** A task can write half its output, then die.
- ◆ **The retry appends.** Now you have one and a half loads.
- ◆ **Truncate and reload is accidentally safe.** Append and merge are not.

## THE CONNECTION

You saw this in module four. Truncate then insert is unsafe on failure and safe on retry.

# What you actually type

Ask about the second run, not the first.

Below is a task that writes to a warehouse. Assume it fails partway through the write and is retried automatically.

Walk through what the warehouse contains after the retry.

Assume the first attempt committed some of its output.

Then answer:

- Is the result identical to a single clean run?
- If not, what is duplicated and how would I detect it?
- What is the smallest change that makes the retry safe?

Also: does this task depend on the wall clock?

THE BLOCK THAT SURPRISES PEOPLE

A job filtering on 'today', retried after midnight, reads a different day.

Last prompt. What does a backfill actually reproduce?

# What you think a backfill does

## THE EXPECTATION

### Recreates the past

- Same input as that day
- Same lookups as that day
- Same code as that day
- Same output as that day

## WHAT RUNS

### Today, with an old date

- Current state of the source
- Current lookup values
- Current code
- A number nobody can reproduce

## THE TIEBREAKER

Only the date parameter goes back in time. Everything else stays where it is.

# What you actually type

The last prompt in the course, and the least run.

```
Below is a task and its DAG. I need to backfill it for a date
three months ago.
```

```
List everything this task reads that is NOT bounded by the
execution date. Those are the things that will use today's
values rather than the values from that date.
```

```
For each, tell me:
```

- Whether a historical version exists at all
- What would differ in the output if it has changed

```
Then: is the output of this backfill comparable to the original
run, or is it a different number that looks the same?
```

## THE QUESTION THAT MATTERS

That closing question is the one to actually answer before you press the button.

A report is wrong roughly one morning in ten. On the bad mornings, one site is missing. Rerunning the report task fixes it every time.

The DAG shows the report depending on the two loads that feed it.

### What is the most likely cause?

- ☐ A An intermittent failure in one of the two upstream loads
- ☐ B An undeclared dependency on something outside this DAG
- ☐ C A retry duplicating rows and confusing the aggregate
- ☐ D A timezone boundary in the report's date filter

### ◆ PAUSE AND DECIDE

## What is the most likely cause?

- A

 Upstream failure  
*A failed upstream task would block the report, not let it run with partial data.*
- B

 Undeclared dependency  
*Rerunning later fixes it. That is a race, and a race means an edge nobody drew.*
- C

 Retry duplication  
*Duplication adds rows. This is a missing site, which is the opposite symptom.*
- D

 Timezone boundary  
*Would cluster around month or day boundaries. One morning in ten is not that pattern.*

### THE SIGNAL

Fixed by rerunning, with nothing else changed, means the input was simply not ready the first time.

The dependency prompt finds four undeclared reads. Three are reference tables that change once a year. One is a dimension loaded daily by another team.

Adding cross-DAG dependencies is a two week piece of work.

### What do you do first?

- ☐ A Declare all four properly. Do it once and do it right
- ☐ B Declare the daily one. Document the other three
- ☐ C Add a sensor that waits for the daily dimension
- ☐ D Move the report later in the day to avoid the race

### ◆ PAUSE AND DECIDE

## What do you do first?

- A

 Declare all four  
*Two weeks of work, and three of the four carry almost no risk. Poor use of the finding.*
- B

 Daily one, document rest  
*Right prioritisation and it leaves the actual race in place while you build.*
- C

 Sensor on the dimension  
*Closes today's race in an afternoon. Then do B properly with the pressure off.*
- D

 Move it later  
*Buys time and does not remove the race. The other team will get slower eventually.*

### THE SIGNAL

Frequency of change is your risk ranking. Annual reference data is not what is breaking your report.

Finance asks you to backfill six months of a report because a bug was fixed. The backfill runs clean and the numbers differ from the originals by small amounts.

Nobody can say which set is correct.

### What is the underlying problem?

- ☐ A The bug fix changed the numbers, which is expected
- ☐ B The dimensions have changed, so neither run is reproducible
- ☐ C The backfill ran with the wrong execution dates
- ☐ D The original runs were wrong and the backfill is right

### ◆ PAUSE AND DECIDE

## What is the underlying problem?

- A

 The fix changed them  
*Partly true and it does not explain why nobody can tell which is right.*
- B

 Dimensions changed  
*Two variables moved at once. You cannot attribute the difference to either.*
- C

 Wrong dates  
*Would produce large obvious differences, not small ones spread evenly.*
- D

 Backfill is right  
*It uses today's dimensions for a period six months ago. That is not obviously right.*

### THE SIGNAL

You changed the code and the reference data at the same time. Neither number is now defensible.

You have applied everything in this course to a pipeline. Reconciled, tested, monitored, dependencies declared, backfill behaviour documented.

A new engineer joins and asks where to start reading.

## What do you hand them?

- ☐ A The original mapping document
- ☐ B The regenerated mapping document
- ☐ C The monitors, since they encode the live assumptions
- ☐ D The list of things nobody could explain

### ◆ PAUSE AND DECIDE

## What do you hand them?

A Original document

*It says what was wanted in 2023. Useful later, misleading as a first read.*

C The monitors

*Genuinely good and narrow. It tells them what is fragile, not what the system is.*

B Regenerated document

*Accurate about behaviour and it reads as settled. It hides every open question.*

D The unexplained list

*The only artifact that shows where knowledge actually ends. Everything else looks complete.*

### THE SIGNAL

Every other document reads as finished. Only that list tells the truth about what nobody knows.

# Beyond SQL, in one line each

| WHERE           | THE ONE THING                                                |
|-----------------|--------------------------------------------------------------|
| Spark types     | The engine will not check your join keys. Assert them        |
| Lazy evaluation | The error is never where the trace points. Paste the lineage |
| Notebooks       | Cell order is in the file. Execution order is not            |
| Dependencies    | Reading a table is a dependency whether declared or not      |
| Retries         | Ask what happens on the second run, not the first            |
| Backfill        | Only the date goes back in time. Everything else stays       |

## THE PATTERN

Different mechanisms, same shape. Something succeeded and the data is wrong.

# Take this to work tomorrow

- ◆ Run the dependency diff on one pipeline. You will find at least one undeclared read.
- ◆ Pick your most important task and ask what a retry does to it.
- ◆ Before the next backfill, list what is not bounded by the execution date.
- ◆ Keep the list of things nobody can explain. It is the most useful document you own.

## THE PAYOFF

The first one takes twenty minutes and finds something in almost every pipeline.

Thirty two prompts. Not one of them decided anything.